

Department of **Cybersecurity Technology Engineering**  
Electrical Engineering Technical College  
Middle Technical University



الحقيبة التعليمية لمادة البرمجة الشيئية للرحلة الثانية  
**Object Oriented Programming**

By

Dr. Loay Talib Al-Saffar\*



\*Education of Dr. Loay Al-Saffar:

Ph.D. in Computer Science, from the University of Potsdam, Department of Computer Science, Germany.

M.Sc. in Computer Science, from the University of Baghdad, Department of Computer Science, Iraq.

B.Sc. in Computer Science, from the University of Baghdad, Department of Computer Science, Iraq.

Dr. Loay Al-Saffar, Email: [loay.alsaffar@mtu.edu.iq](mailto:loay.alsaffar@mtu.edu.iq)

## **Introduction**

Programming methodologies have changed dramatically since the invention of the computer, primarily to accommodate the increasing complexity of programs. For example, when computers were first invented, programming was done by toggling in the binary machine instructions using the computer's front panel. As long as programs were just a few hundred instructions long, this approach worked.

As programs grew, assembly language was invented so that a programmer could deal with larger, increasingly complex programs, using symbolic representations of the machine instructions. As programs continued to grow, high-level languages were introduced that gave the programmer more tools with which to handle complexity. The first widespread language was, of course, FORTRAN. Although FORTRAN was a very impressive first step, it is hardly a language that encourages clear, easy-to-understand programs.

The 1960s gave birth to structured programming. This is the method encouraged by languages such as C and Pascal. The use of structured languages made it possible to write moderately complex programs fairly easily. Structured languages are characterized by their support for stand-alone subroutines, local variables, rich control constructs, and their lack of reliance upon the GOTO. Although structured languages are a powerful tool, even they reach their limit when a project becomes too large.

Consider this: At each milestone in the development of programming, techniques and tools were created to allow the programmer to deal with increasingly greater complexity. Each step of the way, the new approach took the best elements of the previous methods and moved forward. Prior to the invention of OOP, many projects were nearing (or exceeding) the point where the structured approach no longer worked. Object-oriented methods were created to help programmers break through these barriers.

Object-oriented programming took the best ideas of structured programming and combined them with several new concepts. The result was a different way of

organizing a program. In the most general sense, a program can be organized in one of two ways: around its code (what is happening) or around its data (who is being affected). Using only structured programming techniques, programs are typically organized around code. This approach can be thought of as "code acting on data." For example, a program written in a structured language such as C is defined by its functions, any of which may operate on any type of data used by the program.

Object-oriented programs work the other way around. They are organized around data, with the key principle being "data controlling access to code." In an object-oriented language, you define the data and the routines that are permitted to act on that data. Thus, a data type defines precisely what sort of operations can be applied to that data. To support the principles of object-oriented programming, all OOP languages have three traits in common: encapsulation, polymorphism, and inheritance. Let's examine each.

## Encapsulation

*Encapsulation* is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. In an object-oriented language, code and data may be combined in such a way that a self-contained "black box" is created. When code and data are linked together in this fashion, an *object* is created. In other words, an object is the device that supports encapsulation.

Within an object, code, data, or both may be *private* to that object or *public*. Private code or data is known to and accessible only by another part of the object. That is, private code or data may not be accessed by a piece of the program that exists outside the object. When code or data is public, other parts of your program may access it even though it is defined within an object. Typically, the public parts of an object are used to provide a controlled interface to the private elements of the object.

For all intents and purposes, an object is a variable of a user-defined type. It may seem strange that an object that links both code and data can be thought of as a variable. However, in object-oriented programming, this is precisely the case. Each time you define a new type of object, you are creating a new data type. Each specific instance of this data type is a compound variable.

## Polymorphism

Object-oriented programming languages support *polymorphism*, which is characterized by the phrase "one interface, multiple methods." In simple terms, polymorphism is the attribute that allows one interface to control access to a general class of actions. The specific action selected is determined by the exact nature of the situation. A real-world example of polymorphism is a thermostat. No matter what type of furnace your house has (gas, oil, electric, etc.), the thermostat works the same way. In this case, the thermostat (which is the interface) is the same no matter what type of furnace (method) you have. For example, if you want a 70-degree temperature, you set the thermostat to 70 degrees. It doesn't matter what type of furnace actually provides the heat.

This same principle can also apply to programming. For example, you might have a program that defines three different types of stacks. One stack is used for integer values, one for character values, and one for floating-point values. Because of polymorphism, you can define one set of names, **push()** and **pop()**, that can be used for all three stacks. In your program you will create three specific versions of these functions, one for each type of stack, but names of the functions will be the same. The compiler will automatically select the right function based upon the data being stored.

Thus, the interface to a stack—the functions **push()** and **pop()** —are the same no matter which type of stack is being used. The individual versions of these functions define the specific implementations (methods) for each type of data.

Polymorphism helps reduce complexity by allowing the same interface to be used to access a general class of actions. It is the compiler's job to select the *specific action* (i.e., method) as it applies to each situation. You, the programmer, don't need to do this selection manually. You need only remember and utilize the *general interface*.

## Inheritance

*Inheritance* is the process by which one object can acquire the properties of another object. This is important because it supports the concept of *classification*. If you

think about it, most knowledge is made manageable by hierarchical classifications. For example, a Red Delicious apple is part of the classification *apple*, which in turn is part of the *fruit* class, which is under the larger class *food*. Without the use of classifications, each object would have to define explicitly all of its characteristics. However, through the use of classifications, an object need only define those qualities that make it unique within its class. It is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case. As you will see, inheritance is an important aspect of *object-oriented programming*.

## Introducing C++ Classes

In C++, to create an object, you first must define its general form by using the keyword `class`. Here is an example. The following class defines a type called `stack`, which will be used to create a stack:

```
#define SIZE 100
// This creates the class stack.
class stack {
    int stck[SIZE];
    int tos;

public:
    void init();
    void push(int i);
    int pop();
};
```

A `class` may contain private as well as public parts. By default, all items defined in a `class` are private. For example, the variables `stck` and `tos` are private. This means that they cannot be accessed by any function that is not a member of the `class`. This is one way that **encapsulation** is achieved—access to certain items of data may be tightly controlled by keeping them private. Although it is not shown in this example, you can also define private functions, which then may be called only by other members of the `class`.

To make parts of a **class** public (that is, accessible to other parts of your program), you must declare them after the **public** keyword. All variables or functions defined after **public** can be accessed by all other functions in the program. Essentially, the rest of your program accesses an object through its public functions. Although you can have public variables, good practice dictates that you should try to limit their use. Instead, you should make all data private and control access to it through public functions. One other point: Notice that the **public** keyword is followed by a colon.

The functions **init()**, **push()**, and **pop()** are called *member functions* because they are part of the class **stack**. The variables **stck** and **tos** are called *member variables* (or *data members*). Remember, an object forms a bond between code and data. Only member functions have access to the private members of their class. Thus, only **init()**, **push()** , and **pop()** may access **stck** and **tos**.

Once you have defined a **class**, you can create an object of that type by using the class name. In essence, the class name becomes a new data type specifier. For example, this creates an object called **mystack** of type **stack**:

```
stack mystack;
```

When you declare an object of a class, you are creating an *instance* of that class. In this case, **mystack** is an instance of **stack**.

### To review

In C++, **class** creates a new data type that may be used to create objects of that type. Therefore, an object is an instance of a class in just the same way that some other variable is an instance of the **int** data type, for example. Put differently, a class is a logical abstraction, while an object is real. (That is, an object exists inside the memory of the computer.)

The general form of a simple **class** declaration is:

**class** *class-name* {

*private data and functions*

**public:**

*public data and functions*

} *object name list*;

Of course, the *object name list* may be empty.

Inside the declaration of **stack**, member functions were identified using their prototypes. In C++, all functions must be prototyped. Prototypes are not optional. The prototype for a member function within a class definition serves as that function's prototype in general.

When it comes time to actually code a function that is the member of a class, you must tell the compiler which class the function belongs to by qualifying its name with the name of the class of which it is a member.

For example, here is one way to code the **push()** function:

```
void stack::push(int i)
{
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = i;
    tos++;
}
```

The **::** is called the *scope resolution operator*. Essentially, it tells the compiler that this version of **push()** belongs to the **stack** class or, put differently, that this **push()** is in **stack**'s scope. In C++, several different classes can use the same function name. The compiler knows which function belongs to which class because of the scope resolution operator.

When you refer to a member of a class from a piece of code that is not part of the class, you must always do so in conjunction with an object of that class. To do so, use the object's name, followed by the dot operator, followed by the name of the member.

This rule applies whether you are accessing a data member or a function member. For example, this calls `init()` for object `stack1`.

```
stack stack1, stack2;  
stack1.init();
```

This fragment creates two objects, `stack1` and `stack2`, and initializes `stack1`.

Understand that `stack1` and `stack2` are two separate objects. This means, for example, that initializing `stack1` does *not* cause `stack2` to be initialized as well. The only relationship `stack1` has with `stack2` is that they are objects of the same type.

Within a class, one member function can call another member function or refer to a data member directly, without using the dot operator. It is only when a member is referred to by code that does not belong to the class that the object name and the dot operator must be used.

The program shown here puts together all the pieces and missing details and illustrates the `stack` class:

```
#include <iostream>  
using namespace std;  
#define SIZE 100  
  
// This creates the class stack.  
class stack {  
    int stck[SIZE];  
    int tos;  
public:  
    void init();  
    void push(int i);  
    int pop();  
};
```



```

void stack::init(){
    tos = 0;
}
void stack::push(int i){
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = i;
    tos++;
}
int stack::pop(){
    if(tos==0) {
        cout << "Stack underflow.\n";
        return 0;
    }
    tos--;
    return stck[tos];
}

int main()
{
    stack stack1, stack2; // create two stack objects
    stack1.init();
    stack2.init();
    stack1.push(1);
    stack2.push(2);
    stack1.push(3);
    stack2.push(4);
    cout << stack1.pop() << " ";
    cout << stack1.pop() << " ";
    cout << stack2.pop() << " ";
    cout << stack2.pop() << "\n";
    return 0;
}

```

The output from this program is shown here.

3 1 4 2

One last point: Recall that the private members of an object are accessible only by functions that are members of that object. For example, a statement like

```
stack1.tos = 0; // Error, tos is private.
```

could not be in the `main()` function of the previous program because `tos` is *private*.

### So what are Classes and Objects?

In C++, the mechanism that allows you to combine data and the operations on that data in a single unit is called a **class**.

A **class** is a blueprint that specifies the attributes and behaviour of an Object.

Let's assume we are to create a new class human, we can specify some attributes that are specific to human beings:

```
//We are creating a class with attributes name and age.
```

```
class human {  
public:  
    char name[20];  
    int age;  
};
```

An **Object** is an instance of a class, an instance in the sense that it can assume any property or attribute from a class. To make things clear we humans are instances of our genes.

An **Object** is an instantiation (مثيل) of a **class**. (In terms of variables, a class would be the type, and an object would be the variable.)

Using the Class we created earlier, we can create an object from it and then assign the attributes (also called: data members or variables) name and age a value:

```

#include <iostream.h>

class human {
public:
    char name[20];
    int age;
};

int main () {
    human A;

    cout<<" Enter Name and Age: "<<endl;
    cin.getline (A.name, 20);
    cin>>A.age;

    cout <<" Name is "<< A.name << endl;
    cout <<" Age is "<< A.age << endl;

    return 0;
}

```

We created a class with attributes name and age. We have successfully created a Class and we have also created an object and assigned that object attributes from the parent class.

Note that a class is not limited to just one object, you can create as much object as you want:

```

#include <iostream.h>

class human {
public:
    char name[20];
    int age;
};

```

```

int main () {
human A;
human B;
cout<<" Name and Age of Object A: ";
cin.getline (A.name, 20);
cin>>A.age;
cin.ignore();
cout<<" Name and Age of Object B: ";
cin.getline (B.name, 20);
cin>>B.age;
cout <<" Name of object A is "<< A.name << endl;
cout <<" Age of object A is "<< A.age << endl;
cout <<" Name of object B is "<< B.name << endl;
cout <<" Age of object B is "<< B.age << endl;
return 0;
}

```

**Behaviours** (also called: **member functions**, **methods**, **operations**) are actions that an object can perform. For example, a human can run, eat, sleep and so on.

```

#include <iostream.h>

class human {
public:
char name[20];
int age;
void run () {
cout << name <<" is running" << endl;
}
};

int main () {
human A;
cout<<" Enter Name: ";

```

```

cin.getline (A.name, 20);
cout<<" Enter Age: ";
cin>>A.age;
cout <<" Name is "<< A.name << endl;
cout <<" Age is "<< A.age << endl;
A.run();
return 0;
}

```

Another definition of a **class**: it is a collection of a fixed number of components. The components of a class are called the members of the class. That is, a member of a class can be either a variable (to store data) or a function.

- If a member of a class is a variable, you declare it just like any other variable. Also, in the definition of the class, you cannot initialize a variable when you declare it.
- If a member of a class is a function, you typically use the function prototype to declare that member.

## **C++Access Specifiers**

In C++, private, protected, and public are reserved words and are called member access specifiers.

Following are some facts about access specifiers :

- **public** - members are accessible from outside the class
- **private** - members cannot be accessed (or viewed) from outside the class
- **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about Inheritance later.

```

class MyClass {
    public:    // Public access specifier
        int x;    // Public attribute
    private:  // Private access specifier
        int y;    // Private attribute
};

int main() {
    MyClass myObj;
    myObj.x = 25;    // Allowed (public)
    myObj.y = 50;    // Not allowed (private)
    return 0;
}

```

### المحاضرة الأولى والثانية العملية

المحاضرة الأولى العملية تتضمن مراجعة أساسيات اللغة البرمجية وخصوصا موضوع الـ **user defined function** , ومن ثم في المحاضرة الثانية يتم تطبيق أمثلة برمجية لتعريف الطالب بكيفية التعامل مع الـ **classes and objects**

### ملحق المحاضرة الأولى:

## Abstraction

The principle of *abstraction* is easiest to understand through a real-world analogy. A television is a simple piece of technology that can be found in most homes. You are probably familiar with its features: you can turn it on and off, change the channel, adjust the volume, and add external components such as speakers, VCRs, and DVD players. However, can you explain how it works inside the black box? That is, do you know how it receives signals over the air or through a cable, translates them, and displays them on the screen? We certainly can't explain how a television works, yet we are quite capable of using it.

That is because the television clearly separates its internal *implementation* from its external *interface*. We interact with the television through its interface: the power button, channel changer, and volume control.

We don't know, nor do we care, how the television works; we don't care whether it uses a cathode ray tube or some sort of alien technology to generate the image on our screen. It doesn't matter because it doesn't affect the interface.

## **Benefiting from Abstraction**

The abstraction principle is similar in software. You can use code without knowing the underlying implementation. As a trivial example, your program can make a call to the ***sqrt()*** function declared in the header file ***<cmath>*** without knowing what algorithm the function actually uses to calculate the square root. In fact, the underlying implementation of the square root calculation could change between releases of the library, and as long as the interface stays the same, your function call will still work. The principle of abstraction extends to classes as well, you can use the ***cout*** object of class ***ostream*** to stream data to standard output like this:

***cout << "This call will display this line of text\n";***

In this line, you use the documented interface of the ***cout*** insertion operator with a character array. However, you don't need to understand how ***cout*** manages to display that text on the user's screen. You need only know the public interface. The underlying implementation of ***cout*** is free to change as long as the exposed behavior and interface remain the same.

## **Incorporating Abstraction In Your Design**

You should design functions and classes so that you and other programmers can use them without knowing, or relying on, the underlying implementations. The key to abstraction is effectively separating the *interface* from the *implementation*. Implementation is the code you're writing to accomplish the task you set out to accomplish. Interface is the way that other people use your code. In C, the header file that describes the functions in a library you've written is an interface. In object-oriented programming, the interface to a class is the collection of publicly accessible properties and behaviors.

The question of how other programmers will interact with your objects comes into play when designing a class. In C++, a class's properties and behaviors can each be

Dr.Loay Al-Saffar, Email: [loay.alsaffar@mtu.edu.iq](mailto:loay.alsaffar@mtu.edu.iq)

**public**, **protected**, or **private**. Public means that other code can access the property or behavior. protected means that other code cannot. private is a stricter control, which means that not only are the properties or behaviors locked for other code, but even subclasses can't access them.

Designing the exposed interface is all about choosing what to make public. When working on a large project with other programmers, you should view the exposed interface design as a process.

The first step in designing an exposed interface is to consider for whom you are designing it. Is your audience another member of your team? Is this an interface that you will personally be using? Is it something that a programmer external to your company will use? Perhaps a customer or an off-shore contractor? In addition to determining who will be coming to you for help with the interface, this should shed some light on some of your design goals.

If the interface is for your own use, you probably have more freedom to iterate on the design. As you're making use of the interface, you can change it to suit your own needs. However, you should keep in mind that roles on an engineering team change and it is quite likely that, some day, others will be using this interface as well.

Designing an interface for other internal programmers to use is slightly different. In a way, your interface becomes a contract with them. For example, if you are implementing the data store component of a program, others are depending on that interface to support certain operations. You will need to find out all of the things that the rest of the team will be using your class to do. Do they need versioning? What types of data can they store? As a contract, you should view the interface as slightly less flexible. If the interface is agreed upon before coding begins, you'll receive some groans from other programmers if you decide to change it after code has been written.

If the client is an external customer, you will be designing with a very different set of requirements.



## Constructors and Destructors

It is very common for some part of an object to require initialization before it can be used. For example, think back to the `stack` class developed earlier. Before the stack could be used, `tos` had to be set to zero. This was performed by using the function `init()`. Because the requirement for initialization is so common, C++ allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor function.

A *constructor function* is a special function that is a member of a class and has the same name as that class. For example, here is how the `stack` class looks when converted to use a constructor function for initialization:

```
// This creates the class stack.
class stack {
int stck[SIZE];
int tos;

public:
stack(); // constructor
void push(int i);
int pop();
};
```

Notice that the constructor `stack()` has no return type specified. In C++, constructor functions cannot return values and, thus, have no return type.

The `stack()` function is coded like this:

```
// stack's constructor function
stack::stack()
{
tos = 0;
cout << "Stack Initialized\n";
```

```
}
```

Keep in mind that the message **Stack Initialized** is output as a way to illustrate the constructor. In actual practice, most constructor functions will not output or input anything. They will simply perform various initializations.

An object's constructor is automatically called when the object is created. This means that it is called when the object's declaration is executed.

The complement of the constructor is the *destructor*. In many circumstances, an object will need to perform some action or actions when it is destroyed. There are many reasons why a destructor function may be needed. For example, an object may need to deallocate memory that it had previously allocated or it may need to close a file that it had opened. In C++, it is the destructor function that handles deactivation events. The destructor has the same name as the constructor, but it is preceded by a ~. like constructor functions, destructor functions do not have return values.

To see how constructors and destructors work, here is a new version of the `stack` program examined earlier in this chapter. Observe that `init()` is no longer needed.

```
// Using a constructor and destructor.
#include <iostream>
using namespace std;
#define SIZE 100

// This creates the class stack.
class stack {
    int stck[SIZE];
    int tos;

public:
    stack(); // constructor
    ~stack(); // destructor
    void push(int i);
    int pop();
```

```

};

// stack's constructor function
stack::stack(){
    tos = 0;
    cout << "Stack Initialized\n";
}

// stack's destructor function
stack::~~stack() {
    cout << "Stack Destroyed\n";
}

void stack::push(int i){
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = i;
    tos++;
}

int stack::pop(){
    if(tos==0) {
        cout << "Stack underflow.\n";
        return 0;
    }
    tos--;
    return stck[tos];
}

int main(){
    stack a, b; // create two stack objects
    a.push(1);
    b.push(2);
    a.push(3);

```

```

b.push(4);
cout << a.pop() << " ";
cout << a.pop() << " ";
cout << b.pop() << " ";
cout << b.pop() << "\n";
return 0;
}

```

### To review:

Constructor in C++ is a special method that is invoked automatically at the time an object of a class is created. It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class or structure. It constructs the values i.e. provides data for the object which is why it is known as a constructor.

## Some Types of Constructors:

1. **Default Constructors:** Default constructor is the constructor which doesn't take any argument. It has no parameters.

### Example:

```

#include <iostream.h>
class construct {
public:
int a, b;
// Default Constructor
construct() { a = 10;
    b = 20; }
};

int main() {
// Default constructor called automatically
// when the object is created
construct c;
cout << "a: " << c.a << endl << "b: " << c.b;
return 1;
}

```

**Output:**

a: 10  
b: 20

2. **Parameterized Constructors:** It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a Parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

**Example:**

```
#include <iostream.h>

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1) {
        x = x1;
        y = y1;
    }
    int getX() {
        return x;
    }
    int getY() {
        return y;
    }
};

int main()
{
    // Constructor called
    Point p1(10, 15);

    // Access values assigned by constructor
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();

    return 0;
}
```

**Output:**

p1.x = 10, p1.y = 15

3. **Copy Constructor**: A copy constructor is a member function which initializes an object using another object of the same class. A copy constructor has the following general function prototype: `ClassName (const ClassName &old_obj);`

**Example:**

```
#include <iostream.h>

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1) { x = x1; y = y1; }
    // Copy Constructor
    Point(const Point &p1) {x = p1.x; y = p1.y; }
    int getX() { return x; }
    int getY() { return y; }
};

int main() {
    Point p1(10, 15); // Parameterized constructor is called here
    Point p2 = p1; // Copy constructor is called here
    // Let us access values assigned by constructors
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    cout << "\np2.x = " << p2.getX() << ", p2.y = " << p2.getY();
    return 0;
}
```

**Output:**

p1.x = 10, p1.y = 15

p2.x = 10, p2.y = 15

### Notes:

- When an object is declared in a parameterized constructor, the initial values have to be passed as arguments to the constructor function.
- We can have more than one constructor in a class and it is called **Constructor Overloading** (we will talk about overloading later).
- When we create our own copy constructor, we pass an object by reference and we generally pass it as a **const** reference. One reason for passing **const** reference is, we should use **const** in C++ wherever possible so that objects are not accidentally modified.

### المحاضرة الثالثة العملية

تطبيق ما ورد من مثال برمجي أعلاه أو ما يرادفه في المختبر لتعريف الطالب بكيفية التعامل مع الـ **Constructor**

### المحاضرة الرابعة والخامسة النظرية:

## **Inheritance**

As stated earlier, inheritance is one of the major traits of an object oriented programming language. In C++, inheritance is supported by allowing one class to incorporate another class into its declaration. Inheritance allows a hierarchy of classes to be built, moving from most general to most specific. The process involves first defining a *base class*, which defines those qualities common to all objects to be derived from the base. The base class represents the most general description. The classes derived from the base are usually referred to as *derived classes*. A derived class includes all features of the generic base class and then adds qualities specific to the derived class. To demonstrate how this works, the next example creates classes that categorize different types of buildings.

To begin, the *building* class is declared, as shown here. It will serve as the base for *two derived classes*.

```
class building {  
    int rooms;  
    int floors;  
    int area;
```

```

public:
void set_rooms(int num);
int get_rooms();
void set_floors(int num);
int get_floors();
void set_area(int num);
int get_area();

};

```

Because (for the sake of this example) all buildings have three common features—one or more rooms, one or more floors, and a total area—the **building** class embodies these components into its declaration. The member functions beginning with **set** set the values of the private data. The functions starting with **get** return those values.

You can now use this broad definition of a building to create derived classes that describe specific types of buildings. For example, here is a derived class called *house*:

```

// house is derived from building
class house : public building {
int bedrooms;
int baths;
public:
void set_bedrooms(int num);
int get_bedrooms();
void set_baths(int num);
int get_baths();
};

```

Notice how *building* is inherited. The general form for inheritance is:

```

class derived-class : access base-class {

// body of new class

}

```

Here, *access* is optional. However, if present, it must be **public**, **private**, or **protected**. (These options are further examined later) For now, all inherited classes will use

Dr.Loay Al-Saffar, Email: [loay.alsaffar@mtu.edu.iq](mailto:loay.alsaffar@mtu.edu.iq)



**public**. Using **public** means that all of the public members of the base class will become public members of the derived class. Therefore, the public members of the class **building** become public members of the derived class **house** and are available to the member functions of **house** just as if they had been declared inside **house**.

However, **house**'s member functions *do not* have access to the private elements of **building**. This is an important point. Even though **house** inherits **building**, it has access only to the public members of **building**. In this way, inheritance does not circumvent the principles of encapsulation necessary to OOP.

***Remember:** A derived class has direct access to both its own members and the public members of the base class.*

Here is a program illustrating inheritance. It creates two derived classes of **building** using inheritance; one is **house**, the other, **school**.

```
#include <iostream>
using namespace std;
class building {
    int rooms;
    int floors;
    int area;
public:
    void set_rooms(int num);
    int get_rooms();
    void set_floors(int num);
    int get_floors();
    void set_area(int num);
    int get_area();
};

// house is derived from building
class house : public building {
    int bedrooms;
    int baths;
```

```

public:
void set_bedrooms(int num);
int get_bedrooms();
void set_baths(int num);
int get_baths();
};

// school is also derived from building
class school : public building {
int classrooms;
int offices;
public:
void set_classrooms(int num);
int get_classrooms();
void set_offices(int num);
int get_offices();
};

void building::set_rooms(int num) {
rooms = num; }

void building::set_floors(int num) {
floors = num; }

void building::set_area(int num) {
area = num; }

int building::get_rooms() {
return rooms; }

int building::get_floors() {
return floors; }

int building::get_area() {
return area; }

```

```

void house::set_bedrooms(int num){
    bedrooms = num; }

void house::set_baths(int num){
    baths = num; }

int house::get_bedrooms(){
    return bedrooms; }

int house::get_baths(){
    return baths; }

void school::set_classrooms(int num){
    classrooms = num; }

void school::set_offices(int num){
    offices = num; }

int school::get_classrooms(){
    return classrooms; }

int school::get_offices(){
    return offices; }

int main(){
    house h;
    school s;
    h.set_rooms(12);
    h.set_floors(3);
    h.set_area(4500);
    h.set_bedrooms(5);
    h.set_baths(3);
    cout << "house has " << h.get_bedrooms();

```

```

cout << " bedrooms\n";
s.set_rooms(200);
s.set_classrooms(180);
s.set_offices(5);
s.set_area(25000);
cout << "school has " << s.get_classrooms();
cout << " classrooms\n";
cout << "Its area is " << s.get_area();
return 0;
}

```

The output produced by this program is shown here.

```

house has 5 bedrooms
school has 180 classrooms
Its area is 25000

```

As this program shows, the major advantage of inheritance is that you can create a general classification that can be incorporated into more specific ones. In this way, each object can precisely represent its own subclass.

When writing about C++, the terms *base* and *derived* are generally used to describe the inheritance relationship. However, the terms *parent* and *child* are also used. You may also see the terms *superclass* and *subclass*.

## So what is Inheritance?

**Inheritance** is one of the most important concepts in object-oriented programming. it allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code functionality and fast implementation time.

When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class. This existing class is called the **base** class, and the new class is referred to as the **derived** class.

*"Inheritance means access the properties and features of one class into another"*

class. The class who is going to provide its features to another class will be called **base class** and the class who is using the properties and features of another class will be called **derived class**.”

## More on Base and Derived Classes

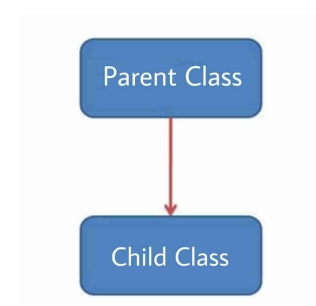
A class can be derived from more than one classes, which means it can inherit data and functions from multiple base classes. To define a derived class, we use a class derivation list to specify the base class(es). A class derivation list names one or more base classes and has the form:

**class derived-class:** access-specifier **base-class**

Where *access-specifier*, also known as *access*, is one of **public**, **protected**, or **private**, and *base-class* is the name of a previously defined class. If the access-specifier is not used, then it is **private** by default.

Consider a base class **Shape** and its derived class **Rectangle** as follows:

```
#include <iostream>
using namespace std;
// Base class (also known as Parent class)
class Shape {
public:
    void setWidth(int w) {
        width = w;
    }
    void setHeight(int h) {
        height = h;
    }
protected:
    int width;
    int height;
};
```



```
// Derived class (also known as child class)
class Rectangle: public Shape {
public:
    int getArea() {
        return (width * height);
    }
};

int main() {
    Rectangle Rect;
    Rect.setWidth(5);
    Rect.setHeight(7);
    // Print the area of the object.
    cout << "Total area: " << Rect.getArea() << endl;
    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Total area: 35
```

When deriving a class from a base class, the base class may be inherited through **public**, **protected** or **private** inheritance. The type of inheritance is specified by the access-specifier as explained above. We hardly use **protected** or **private** inheritance, but **public** inheritance is commonly used.

While using different type of inheritance, following rules are applied:

- **Public Inheritance** – When deriving a class from a **public** base class, **public** members of the base class become **public** members of the derived class and **protected** members of the base class become **protected** members of the derived class. A base class's **private** members are never accessible directly from a derived class, but can be accessed through calls to the **public** and **protected** members of the base class.
- **Protected Inheritance** – When deriving from a **protected** base class, **public** and **protected** members of the base class become **protected** members of the derived class.
- **Private Inheritance** – When deriving from a **private** base class, **public** and **protected** members of the base class become **private** members of the derived class.

**Note:** The private members in the base class cannot be directly accessed in the derived class, while protected members can be directly accessed.

**For example,** Classes B, C and D all contain the variables x, y and z in below example. It is just question of access:

```
// C++ Implementation to show that a derived class
// doesn't inherit access to private data members.
// However, it does inherit a full parent object
class A
{
public:
    int x;
protected:
    int y;
private:
    int z;
};
class B : public A
{
    // x is public
    // y is protected
    // z is not accessible from B
};
class C : protected A
{
    // x is protected
    // y is protected
    // z is not accessible from C
};
class D : private A // 'private' is default for classes
{
    // x is private
    // y is private
    // z is not accessible from D
};
```

المحاضرة الرابعة والخامسة العملية: تطبيق امثلة برمجية عن الـ Inheritance وتمييز الاختلاف باستخدام الـ access-specifier

Dr.Loay Al-Saffar, Email: [loay.alsaffar@mtu.edu.iq](mailto:loay.alsaffar@mtu.edu.iq)

## Some Types of Inheritance

### 1- Single Inheritance

A derived class with only one base class is called **single inheritance**.

#### Example:

```
#include<iostream.h>

class Employee {
    int Id;
    char Name[25];
    int Age;
    long Salary;

public:
    void GetData() {
        cout<<"\n\tEnter Employee Id : ";
        cin>>Id;
        cin.ignore();
        cout<<"\n\tEnter Employee Name : ";
        cin.getline(Name, 25);
        cout<<"\n\tEnter Employee Age : ";
        cin>>Age;
        cout<<"\n\tEnter Employee Salary : ";
        cin>>Salary;
    }

    void PutData() {
        cout<<"\n\nEmployee Id : "<<Id;
        cout<<"\nEmployee Name : "<<Name;
        cout<<"\nEmployee Age : "<<Age;
        cout<<"\nEmployee Salary : "<<Salary;
    }
};
```

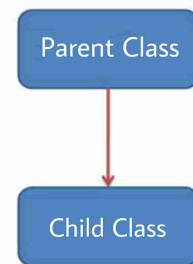


```

class Company : public Employee //Statement 1: Derived Class
{
int RegNo;
char CName[25];
public:
void ReadData() {
cout<<"\n\tEnter Registration No. : ";
cin>>RegNo;
cin.ignore();
cout<<"\n\tEnter Company Name : ";
cin.getline(CName,25);
}
void WriteData() {
cout<<"\n\nRegistration No. : "<<RegNo;
cout<<"\nCompany Name : "<<CName<<endl;
}
};

int main() {
Company C;      //Statement 2: Creating Object of Derived Class
C.GetData();    //Statement 3: Calling Base Class Method()
C.ReadData();   //Statement 4: Calling Derived Class Method()
C.PutData();    //Statement 5: Calling Base Class Method()
C.WriteData();  //Statement 6: Calling Derived Class Method()
return 0;
}

```



### Output:

```

Enter Employee Id : 1
Enter Employee Name : Ahmed
Enter Employee Age : 25
Enter Employee Salary : 500
Enter Registration No. : 715
Enter Company Name : Lomnam

```

```

Employee Id : 1
Employee Name : Ahmed
Employee Age : 25
Employee Salary : 500
Registration No. : 715
Company Name : Lomnam

```

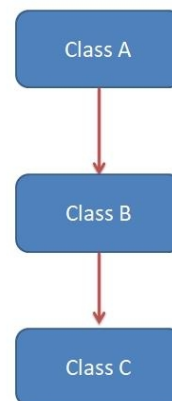
Consider the statement 1, we are publically inheriting an **Employee** class into **Company** class using colon(:).

Now, The object of **Company** class can access the member function GetData() and PutData() of **Employee** class.

## 2. Multi-Level Inheritance:

In C++ programming, not only can you derive a class from the base class but you can also derive a class from the derived class. This form of inheritance is known as *multilevel* inheritance.

```
#include <iostream>
using namespace std;
// single base class
class A {
public:
    int a;
    void get_A_data()
    {
        cout << "Enter value of a: ";
        cin >> a;
    }
};
// derived class from base class
class B : public A {
public:
    int b;
    void get_B_data() {
        cout << "Enter value of b: ";
        cin >> b;
    }
};
// derived from class derive1
class C : public B {
private:
    int c;
public:
    void get_C_data() {
        cout << "Enter value of c: ";
        cin >> c;
    }
}
```



```

    // function to print sum
    void sum()
    {
        int ans = a + b + c;
        cout << "sum: " << ans;
    }
};

int main() {
    // object of sub class
    C obj;

    obj.get_A_data();
    obj.get_B_data();
    obj.get_C_data();
    obj.sum();
    return 0;
}

```

### Output:

```

Enter value of a: 4
Enter value of b: 5
Enter value of c: 9
sum: 18

```

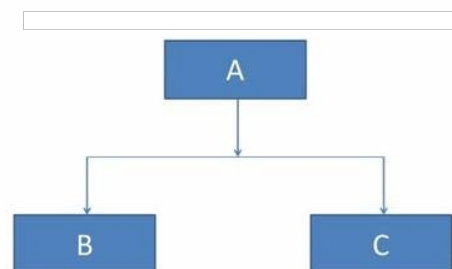
### 3- Hierarchical Inheritance:

In this type of inheritance, more than one derived class is created from a single base class.

```

#include <iostream.h>
// base class
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle" << endl;
    }
};
// first sub class
class Car: public Vehicle { };

```



```

// second sub class
class Bus: public Vehicle { };

int main() {
    // creating object of sub class will
    // invoke the constructor of base class
    Car obj1;
    Bus obj2;
    return 0;
}

```

### Output:

This is a Vehicle  
This is a Vehicle

## Function Overloading

One way that C++ achieves polymorphism is through the use of function overloading. In C++, two or more functions can share the same name as long as their parameter declarations are different. In this situation, the functions that share the same name are said to be *overloaded*, and the process is referred to as *function overloading*.

To see why function overloading is important, first consider two functions defined by the C subset: `abs()`, and `fabs()`. The `abs()` function returns the absolute value of an integer, and `fabs()` returns the absolute value of a **double**. Although these functions perform almost identical actions, in C two slightly different names must be used to represent these essentially similar tasks. This makes the situation more complex, conceptually, than it actually is. Even though the underlying concept of each function is the same, the programmer has to remember two things, not just one. *However*, in C++, you can use just one name for all functions, as this program illustrates:

```

#include <iostream>
using namespace std;

// abs is overloaded three ways
int abs(int i);
double abs(double d);

```

```

int main() {
    cout << abs(-10) << "\n";
    cout << abs(-11.0) << "\n";
    return 0;
}

int abs(int i)
{
    cout << "Using integer abs()\n";
    return i<0 ? -i : i;
}

double abs(double d)
{
    cout << "Using double abs()\n";
    return d<0.0 ? -d : d;
}

```

The output from this program is shown here.

```

Using integer abs()
10
Using double abs()
11

```

This program creates two similar but different functions called **abs()**, each of which returns the absolute value of its argument. The compiler knows which function to call in each situation because of the type of the argument. The value of overloaded functions is that they allow related sets of functions to be accessed with a common name. Thus, the name **abs()** represents the *general action* that is being performed. It is left to the compiler to choose the right *specific method* for a particular circumstance. You need only remember the general action being performed. Due to polymorphism, two things to remember have been reduced to one. This example is fairly trivial, but if you expand the concept, you can see how polymorphism can help you manage very complex programs.

In general, to overload a function, simply declare different versions of it. The compiler takes care of the rest. You must observe one important restriction when overloading a function: the type and/or number of the parameters of each overloaded

Dr.Loay Al-Saffar, Email: [loay.alsaffar@mtu.edu.iq](mailto:loay.alsaffar@mtu.edu.iq)

function must differ. It is not sufficient for two functions to differ only in their return types. They must differ in the types or number of their parameters. (Return types do not provide sufficient information in all cases for the compiler to decide which function to use.) Of course, overloaded functions *may* differ in their return types, too.

### To review:

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but different parameters. When a function name is overloaded with different jobs it is called *Function Overloading*.

In *Function Overloading* "Function" name should be the same and the arguments should be different. Function overloading can be considered as an example of a polymorphism feature in C++.

If multiple functions having same name but parameters of the functions should be different is known as *Function Overloading*. If we have to perform only one operation and having same name of the functions increases the readability of the program.

### **Does function overloading work with Inheritance?**

If we have a function in *base class* and another function with the same name in *derived class*, ***can the base class function be called from derived class object?***

This is an interesting question and as an experiment, predict the output of the following C++ program:

```
#include <iostream>
using namespace std;
class Base {
public:
    int f(int i)
    {
        cout << "f(int): ";
        return i+3;
    }
};
```

```

class Derived : public Base {
public:
    double f(double d) {
        cout << "f(double): ";
        return d+3.3;
    }
};

int main()
{
    Derived D;
    cout << D.f(3) << '\n';
    cout << D.f(3.3) << '\n';
    return 0;
}

```

The output of this program is:

**f(double): 6.3**

**f(double): 6.6**

Instead of the supposed output:

**f(int): 6**

**f(double): 6.6**

Overloading doesn't work for derived class in the C++ programming language. There is no overload resolution between Base and Derived. The compiler looks into the scope of Derived, finds the single function "double f(double)" and calls it. This is name hiding, and not overloading it never disturbs the (enclosing) scope of Base.

In C++, there is no *overloading* across scopes and derived class scopes are not an exception to this general rule, instead it is called *overriding*.

## Function Overriding

Function overriding is a type of polymorphism in which we redefine the member function of a class which it inherited from its base class. The function signature remains same but the working of the function is altered to meet the needs of the

derived class. So, when we call the function using its name for the parent object, the parent class function is executed. But when we call the function using the child object, the child class version will be executed.

***So function Overriding (achieved at run time) is the redefinition of base class function in its derived class with same signature (i.e. return type and parameters).***

| <b><u>Function Overloading</u></b>                                                                             | <b><u>Function Overriding</u></b>                                                                           |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Function Overloading provides multiple definitions of the function by changing signature.                      | Function Overriding is the redefinition of base class function in its derived class with same signature.    |
| An example of compile time polymorphism.                                                                       | An example of run time polymorphism.                                                                        |
| Function signatures should be different.                                                                       | Function signatures should be the same.                                                                     |
| Overloaded functions are in same scope.                                                                        | Overridden functions are in different scopes.                                                               |
| Overloading is used when the same function has to behave differently depending upon parameters passed to them. | Overriding is needed when derived class function has to do some different job than the base class function. |
| A function has the ability to load multiple times.                                                             | A function can be overridden only a single time.                                                            |
| In function overloading, we don't need inheritance.                                                            | In function overriding, we need an inheritance concept.                                                     |

Polymorphism also encompasses *Operator Overloading*, which you may explore further on your own if it aligns with your interests.

#### المحاضرة السادسة والسابعة العملية

القيام بتطبيق أمثلة برمجية من تصميم الطالب لأنواع الـ **inheritance** مع استخدام **function overriding**



## Pointers

C++ provides two pointer operators, which are (a) Address of Operator & and (b) Indirection Operator \*.

A pointer is a variable that contains the address of another variable or you can say that a variable that contains the address of another variable is said to "point to" the other variable. A variable can be any data type including an object, structure or again pointer itself.

The . (dot) operator and the -> (arrow) operator are used to reference individual members of classes.

### The Address of Operator &

The & is a unary operator that returns the memory address of its operand. For example, if **var** is an integer variable, then **&var** is its address. This operator has the same precedence and right-to-left associativity as the other unary operators.

You should read the & operator as "the address of" which means **&var** will be read as "the address of var".

### The Indirection Operator \*

The second operator is indirection Operator \*, and it is the complement of &. It is a unary operator that returns the value of the variable located at the address specified by its operand.

The following program executes the two operations:

```
#include <iostream.h>
int main () {
    int var;
    int *ptr;
    int val;
    var = 3000;
    ptr = &var; // take the address of var
    val = *ptr; // take the value available at ptr
    cout << "Value of var :" << var << endl;
    cout << "Value of ptr :" << ptr << endl;
    cout << "Value of val :" << val << endl;
    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Value of var :3000
```

```
Value of ptr :0xbff64494
```

```
Value of val :3000
```

## C++ Dynamic Memory

You can allocate memory at run time for the variable of a given type using a special operator in C++ which returns the address of the space allocated. This operator is called **new** operator.

If you are not in need of dynamically allocated memory anymore, you can use **delete** operator, which de-allocates memory that was previously allocated by new operator.

### new and delete Operators

There is following generic syntax to use new operator to allocate memory *dynamically* for any data-type.

**new data-type;**

Here, data-type could be any built-in data type including an array or any user defined data types include class or structure. Let us start with built-in data types. For example we can define a pointer to type **double** and then request that the memory be allocated at execution time. We can do this using the **new** operator with the following statements:

```
double* pvalue; // Pointer
```

```
pvalue = new double; // Request memory for the variable
```

At any point, when you feel a variable that has been dynamically allocated is not any more required, you can free up the memory that it occupies in the free store with the 'delete' operator as follows:

```
delete pvalue; // Release memory pointed to by pvalue
```

Let us put above concepts and form the following example to show how 'new' and 'delete' work:

```
#include <iostream.h>
```

```
int main () {
```

```
int* pvalue; // Pointer
```

```

pvalue = new int; // Request memory for the variable
*pvalue = 29495; // Store value at allocated address
cout << "Value of pvalue : " << *pvalue << endl;

delete pvalue; // free up the memory.
return 0;
}

```

If we compile and run above code, this would produce the following result:  
Value of pvalue : 29495

## Dynamic Memory Allocation for Arrays

Consider you want to allocate memory for an array of characters, i.e., string of 20 characters. Using the same syntax what we have used above we can allocate memory dynamically as shown below.

```

char* pvalue; // Pointer
pvalue = new char[20]; // Request memory for the variable
To remove the array that we have just created the statement would look like this –
delete [] pvalue; // Delete array pointed to by pvalue

#include<iostream.h>
int main() {
int x, n;
cout << "Enter the number of characters:" << "\n";
cin >>n;
char *arr = new char[n];
cout << "Enter " << n << " characters" << endl;
for (x = 0; x < n; x++) {
    cin >> arr[x];
}
cout << "You entered: ";
for (x = 0; x < n; x++) {
    cout << arr[x] << " ";
}
cout<<endl<<"If you had entered more than "<< n <<" characters,
then the rest has been ignored"<<endl;
return 0;
}

```

Following the similar generic syntax of **new** operator, you can allocate for a multi-dimensional array as follows:

```
double** pvalue; // Pointer
```

```
pvalue = new double [3][4]; // Allocate memory for a 3x4 array
```

However, the syntax to release the memory for multi-dimensional array will still remain same as above:

```
delete [] pvalue; // Delete array pointed to by pvalue
```

## Dynamic Memory Allocation for Objects

Objects are no different from simple data types. For example, consider the following code where we are going to use **an array of objects** to clarify the concept:

```
#include <iostream>
```

```
using namespace std;
```

```
class Box {
```

```
public:
```

```
Box() {
```

```
cout << "Constructor called!" <<endl;
```

```
}
```

```
};
```

```
int main() {
```

```
int x;
```

```
cout << "how many objects do you want? ";
```

```
cin>>x;
```

```
Box* myBoxArray = new Box[x];
```

```
delete [] myBoxArray; // Delete array
```

```
return 0;
```

```
}
```

If you were to allocate an array of **x** Box objects, the Simple constructor would be called four times.

If we compile and run above code, and enter **x = 4**, this would produce the following result:

Constructor called!  
Constructor called!  
Constructor called!  
Constructor called!

This program will explain how to read and print multiple student details using also

### Array of Objects.

```
/*C++ program to create student class, read and print N student's
details
(Example of array of objects).*/
#include <iostream>
using namespace std;
class student {
private:
char name[30];
int idNo;
int total;
float perc;
public:
//member function to get student's details
void getDetails(void);
//member function to print student's details
void putDetails(void);
};

//member function definition, outside of the class
void student::getDetails(void) {
cout << "Enter name: " ;
cin.ignore();
cin.getline (name, 30);
cout << "Enter ID number: ";
cin >> idNo;
cout << "Enter total marks outof 500: ";
cin >> total;
perc=(float) total/5;
}
```

```

//member function definition, outside of the class
void student::putDetails(void){
    cout << "Name:"<< name << ", ID Number:" << idNo << ",Total:" <<
    total << ",Percentage:" << perc<<endl;
}

int main() {
    int n,loop;
    cout << "Enter total number of students: ";
    cin >> n;
    student* st = new student[n]; //array of objects creation
    for(loop=0;loop< n; loop++){
        cout << "Enter details of student " << loop+1 << ":\n";
        st[loop].getDetails();
    }
    cout<< endl;
    for(loop=0;loop< n; loop++){
        cout << "\n\nDetails of student " << (loop+1) << ":\n";
        st[loop].putDetails();
    }
    delete [] st;
    return 0;
}

```

## C++ Pointer Arithmetic

As you understood pointer is an address which is a numeric value; therefore, you can perform arithmetic operations on a pointer just as you can a numeric value.

There are four arithmetic operators that can be used on pointers: ++, --, +, and -

To understand pointer arithmetic, let us consider that **ptr** is an integer pointer which points to the address 1000. Assuming 32-bit integers (each integer occupies 4 bytes), let us perform the following arithmetic operation on the pointer **ptr++**

the **ptr** will point to the location 1004 because each time **ptr** is incremented, it will point to the next integer. This operation will move the pointer to next memory

location without impacting actual value at the memory location. If **ptr** points to a character whose address is 1000, then above operation will point to the location 1001 because next character will be available at 1001.

## Incrementing a Pointer

We prefer using a pointer in our program instead of an array because the variable pointer can be incremented, unlike the array name which cannot be incremented because it is a constant pointer. The following program increments the variable pointer to access each succeeding element of the array:

```
#include <iostream>
using namespace std;
const int MAX = 3;
int main () {
    int var[MAX] = {10, 100, 200};
    int *ptr;
    // let us have array address in pointer.
    ptr = var;
    for (int i = 0; i < MAX; i++) {
        cout << "Address of var[" << i << "] = ";
        cout << ptr << endl;
        cout << "Value of var[" << i << "] = ";
        cout << *ptr << endl;
        // point to the next location
        ptr++;
    }
    return 0;
}
```

When the above code is compiled and executed, it produces result something as follows:

```
Address of var[0] = 0xbfa088b0
Value of var[0] = 10
Address of var[1] = 0xbfa088b4
Value of var[1] = 100
Address of var[2] = 0xbfa088b8
Value of var[2] = 200
```

## Decrementing a Pointer

The same considerations apply to decrementing a pointer, which decreases its value by the number of bytes of its data type as shown below:

```
#include <iostream>
using namespace std;
const int MAX = 3;
int main () {
    int var[MAX] = {10, 100, 200};
    int *ptr;
    // let us have address of the last element in pointer.
    ptr = &var[MAX-1];
    for (int i = MAX; i > 0; i--) {
        cout << "Address of var[" << i << "] = ";
        cout << ptr << endl;
        cout << "Value of var[" << i << "] = ";
        cout << *ptr << endl;
        // point to the previous location
        ptr--;
    }
    return 0;
}
```

When the above code is compiled and executed, it produces result something as follows:

```
Address of var[3] = 0xbfdb70f8
Value of var[3] = 200
Address of var[2] = 0xbfdb70f4
Value of var[2] = 100
Address of var[1] = 0xbfdb70f0
Value of var[1] = 10
```

## Pointer Comparisons

Pointers may be compared by using relational operators, such as `==`, `<`, and `>`. If ***p1*** and ***p2*** point to variables that are related to each other, such as elements of the same array, then ***p1*** and ***p2*** can be meaningfully compared.



The following program modifies the previous example one by incrementing the variable pointer so long as the address to which it points is either less than or equal to the address of the last element of the array, which is **&var[MAX - 1]** :

```
#include <iostream>
using namespace std;
const int MAX = 3;
int main () {
int var[MAX] = {10, 100, 200};
int *ptr;
// let us have address of the first element in pointer.
ptr = var;
int i = 0;
while ( ptr <= &var[MAX - 1] ) {
cout << "Address of var[" << i << "] = ";
cout << ptr << endl;
cout << "Value of var[" << i << "] = ";
cout << *ptr << endl;
// point to the previous location
ptr++;
i++;
}
return 0;
}
```

When the above code is compiled and executed, it produces result something as follows:

```
Address of var[0] = 0xbfce42d0
Value of var[0] = 10
Address of var[1] = 0xbfce42d4
Value of var[1] = 100
Address of var[2] = 0xbfce42d8
Value of var[2] = 200
```

## C++ this Pointer

Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions.

Therefore, inside a member function, **this** may be used to refer to the invoking object.

Friend functions do not have a **this** pointer, because friends are not members of a class. Only member functions have a **this** pointer. (*Friend functions will be discussed later*)

Let us try the following example to understand the concept of **this** pointer:

```
#include <iostream>
using namespace std;
class Box {
public:
    // Constructor definition
    Box(double l = 2.0, double b = 2.0, double h = 2.0) {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
    }
    double Volume() {
        return length * breadth * height;
    }
    int compare(Box box) {
        return this-> Volume() > box.Volume();
    }
private:
    double length; // Length of a box
    double breadth; // Breadth of a box
    double height; // Height of a box
};

int main() {
    Box Box1(3.3, 1.2, 1.5); // Declare box1
    Box Box2(8.5, 6.0, 2.0); // Declare box2
    if(Box1.compare(Box2)) {
        cout << "Box2 is smaller than Box1" << endl;
    }
}
```

```

else {
cout << "Box2 is equal to or larger than Box1" <<endl;
}
return 0;
}

```

When the above code is compiled and executed, it produces the following result:

Constructor called.

Constructor called.

Box2 is equal to or larger than Box1

## C++ Null Pointers

It is always a good practice to assign the pointer **NULL** to a pointer variable in case you do not have exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned **NULL** is called a null pointer.

The **NULL** pointer is a constant with a value of zero defined in several standard libraries, including iostream. Consider the following program:

```

#include <iostream>
using namespace std;
int main () {
int *ptr = NULL;
cout << "The value of ptr is " << ptr ;
return 0;
}

```

When the above code is compiled and executed, it produces the following result –

The value of ptr is 0

if a pointer contains the null (zero) value, it is assumed to point to nothing.

To check for a null pointer you can use an **if** statement as follows:

```

if(ptr) // succeeds if p is not null
if(!ptr) // succeeds if p is null

```

Thus, if all unused pointers are given the null value and you avoid the use of a **NULL** pointer, you can avoid the accidental misuse of an uninitialized pointer. Many times, uninitialized variables hold some **junk values** and it becomes difficult to debug the program.

#### المحاضرة الثامنة والتاسعة العملية

القيام بتطبيق أمثلة برمجية عن الـ Pointer والـ Dynamic Memory Allocation for Objects

#### المحاضرة العاشرة والحادية عشر والثانية عشر النظرية:

## Polymorphism and Templates in C++

As discussed earlier, The word **polymorphism** means having many forms. It is considered as one of the important features of **Object Oriented Programming**.

In C++ polymorphism is mainly divided into two types:

- **Compile time Polymorphism** (for example: by using function overloading)

```
// C++ program for function overloading

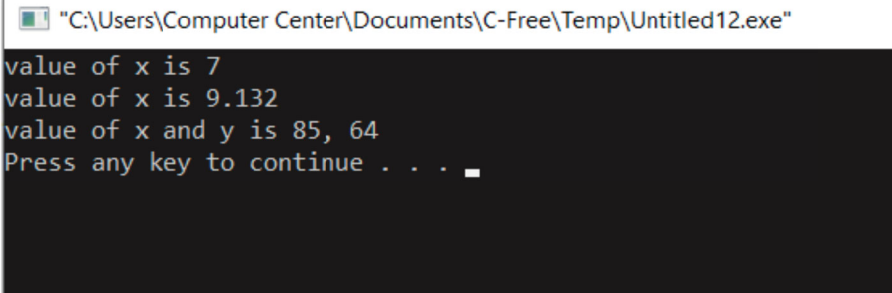
#include <iostream.h>
class Geeks
{
public:
// function with 1 intparameter
void func(intx)
{
cout<< "value of x is " << x << endl;
}
// function with same name but 1 double parameter
void func(double x)
{
cout<< "value of x is " << x << endl;
}
// function with same name and 2 intparameters
void func(intx, inty)
{
cout<< "value of x and y is " << x << ", " << y << endl;
}
};
```

```

int main() {
Geeks obj1;
// Which function is called will depend on the parameters passed
// The first 'func' is called
obj1.func(7);
// The second 'func' is called
obj1.func(9.132);
// The third 'func' is called Output:
obj1.func(85,64);
return 0;
}

```

output:



```

"C:\Users\Computer Center\Documents\C-Free\Temp\Untitled12.exe"
value of x is 7
value of x is 9.132
value of x and y is 85, 64
Press any key to continue . . .

```

In the above example, a single function named **func** acts differently in three different situations which is the property of *polymorphism*.

- **Runtime Polymorphism**: This type of polymorphism is achieved by Function Overriding. (Previous example was given in page 36)

## C++ Templates

Templates are powerful features of C++ which allows you to create a single function or a class to work with different data types. Templates are often used for the purpose of code reusability and flexibility.

The concept of templates can be used in two different ways:

1. *Function Templates*
2. *Class Templates*

## Function Templates

A function template works in a similar to a normal function, with one key difference:

***A single function template can work with different data types at once but, a single normal function can only work with one set of data types.***

Normally, if you need to perform identical operations on two or more types of data, you use function overloading to create two functions with the required function declaration. However, a better approach would be to use *function templates* because you can perform the same task writing less and maintainable code.

### **How to declare a function template?**

A function template starts with the keyword **template** followed by template parameter/s inside < > which is followed by function declaration.

```
template <typename T>
T someFunction(T arg)
{ ...
..
...
}
```

In the above code, **T** is a template argument that accepts different data types (*int*, *float*), and **typename** is a keyword (you can use **class** instead of **typename**).

When, an argument of a data type is passed to someFunction( ), compiler generates a new version of someFunction() for the given data type.

```
// Program to display largest among two inputs using function
templates.
#include <iostream.h>
// template function
template <typename T>
T Large(T n1, T n2) {
return (n1 > n2) ? n1 : n2;
}
```

```

int main() {

int i1, i2;

float f1, f2;

char c1, c2;

cout<< "Enter two integers:\n";

cin>> i1 >> i2;

cout<< Large(i1, i2) <<" is larger." << endl;

cout<< "\nEnter two floating-point numbers:\n";

cin>> f1 >> f2;

cout<< Large(f1, f2) <<" is larger." << endl;

cout<< "\nEnter two characters:\n";

cin>> c1 >> c2;

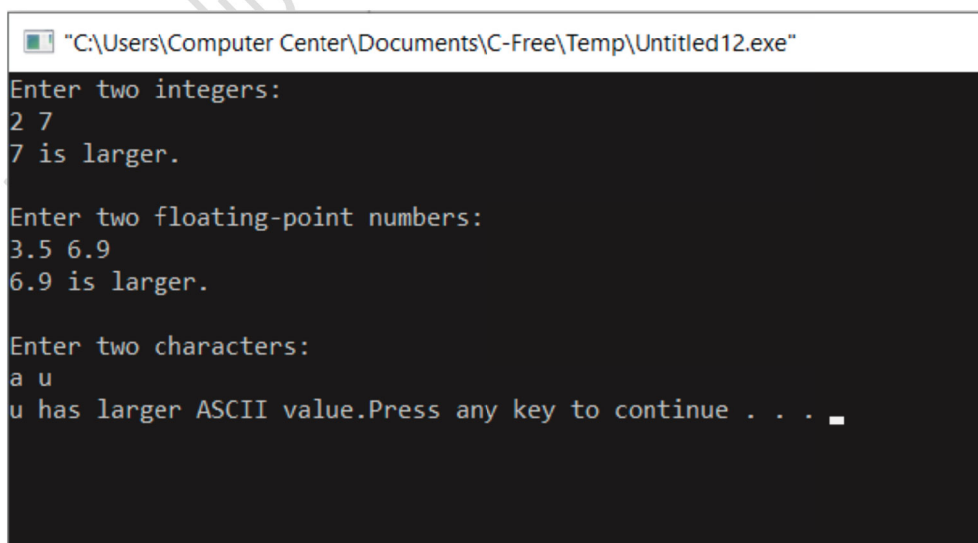
cout<< Large(c1, c2) << " has larger ASCII value.";

return 0;

}

```

### **output:**



```

"C:\Users\Computer Center\Documents\C-Free\Temp\Untitled12.exe"
Enter two integers:
2 7
7 is larger.

Enter two floating-point numbers:
3.5 6.9
6.9 is larger.

Enter two characters:
a u
u has larger ASCII value.Press any key to continue . . .

```

```

// C++ Program to implement Bubble sort
// using template function
#include <iostream>
using namespace std;
// A template function to implement bubble sort.
// We can use this for any data type that supports
// comparison operator < and swap works for it.
template <class T> void bubbleSort(T a[], int n){
    for (int i = 0; i < n - 1; i++)
        for (int j = n - 1; i < j; j--)
            if (a[j] < a[j - 1])
                swap(a[j], a[j - 1]);
}

// Driver Code
int main(){
    int a[5] = { 10, 50, 30, 40, 20 };
    int n = sizeof(a) / sizeof(a[0]);
    // calls template function to sort integers
    bubbleSort<int>(a, n);
    cout << " Sorted array : ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
    char b[5] = { 'f', 'a', 'd', 'i', 'z' };
    n = sizeof(b) / sizeof(b[0]);
    // calls template function to sort charaters
    bubbleSort<char>(b, n);

    cout << " Sorted array : ";
    for (int i = 0; i < n; i++)
        cout << b[i] << " ";
    cout << endl;
    return 0;
}

```

**Output:**

Sorted array : 10 20 30 40 50  
Sorted array : a d f i z



## **Class Templates**

Like function templates, you can also create class templates. Sometimes, you need a class implementation that is same for all classes, only the data types used are different.

class template starts with the keyword **template** followed by template parameter(s) inside < > which is followed by the class declaration.

```
template <class T>
class className {
    private:
        T var;
        ... ..
    public:
        T functionName(T arg);
        ... ..
};
```

In the above declaration, **T** is the template argument which is a place holder for the data type used, and **class** is a keyword. Inside the class body, a member variable **var** and a member function **functionName()** are both of type **T**.

Once we've declared and defined a class template, we can create its objects in other classes or functions (such as the main() function), to create a class template object, you must need to define data type inside a < > when creation:

```
className <Data type> classObject;
```

for example:

```
className <int> classObject;
```

```
className <float> classObject;
```

### **Example 1:**

```
#include <iostream>
using namespace std;
// Class template
template <class T>
class Number {
    private:
        // Variable of type T
        T num;

    public:
        Number(T n) : num(n) {}    // constructor

        T getNum() {
            return num;
        }
};

int main() {
    // create object with int type
    Number<int> numberInt(7);
    // create object with double type
    Number<double> numberDouble(7.7);

    cout << "int Number = " << numberInt.getNum() << endl;
    cout << "double Number = " << numberDouble.getNum() << endl;
    return 0;
}
```

### **Output:**

```
int Number = 7

double Number = 7.7
```

### **Example 2:**

```
#include <iostream>
using namespace std;

template <class T>

class Calculator {

private:

    T num1, num2;

public:

    Calculator(T n1, T n2) {
        num1 = n1;
        num2 = n2;
    }

    void displayResult() {

        cout << "Numbers: " << num1 << " and " << num2 << "." << endl;
        cout << num1 << " + " << num2 << " = " << add() << endl;
        cout << num1 << " - " << num2 << " = " << subtract() << endl;
        cout << num1 << " * " << num2 << " = " << multiply() << endl;
        cout << num1 << " / " << num2 << " = " << divide() << endl;
    }

    T add() { return num1 + num2; }

    T subtract() { return num1 - num2; }

    T multiply() { return num1 * num2; }

    T divide() { return num1 / num2; }

};
```

```

int main() {

    Calculator<int> intCalc(2, 1);

    Calculator<float> floatCalc(2.4, 1.2);

    cout << "Int results:" << endl;

    intCalc.displayResult();

    cout << endl << "Float results:" << endl;

    floatCalc.displayResult();

    return 0;

}

```

### **Output:**

Int results:

Numbers: 2 and 1.

2 + 1 = 3

2 - 1 = 1

2 \* 1 = 2

2 / 1 = 2

Float results:

Numbers: 2.4 and 1.2.

2.4 + 1.2 = 3.6

2.4 - 1.2 = 1.2

2.4 \* 1.2 = 2.88

2.4 / 1.2 = 2

### **Example 3:**

// C++ Program to implement template Array class

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T> class Array {
```

```
private:
```

```
    T* ptr;
```

```
    int size;
```

```
public:
```

```
    Array(T arr[], int s);
```

```
    void print();
```

```
};
```

```
template <typename T> Array<T>::Array(T arr[], int s){
```

```
    ptr = new T[s];
```

```
    size = s;
```

```
    for (int i = 0; i < size; i++)
```

```
        ptr[i] = arr[i];
```

```
}
```

```
template <typename T> void Array<T>::print(){
```

```
    for (int i = 0; i < size; i++)
```

```
        cout << " " << *(ptr + i);
```

```
    cout << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    int arr1[5] = { 1, 2, 3, 4, 5 };
```

```
    Array<int> a(arr1, 5);
```

```
    a.print();
```

```

    char arr2[5] = { '*', '#', 'a', '@', '&' };
    Array<char> b(arr2, 5);
    b.print();
    return 0;
}

```

**Output:**

```

1 2 3 4 5
* # a @ &

```

**What is the difference between function overloading and templates?**

Both function overloading and templates are examples of polymorphism features of OOP. Function overloading is used when multiple functions do quite similar (not identical) operations, templates are used when multiple functions do identical operations.

**Can there be more than one argument for templates?**

Yes, like normal parameters, we can pass more than one data type as arguments to templates. The following example demonstrates the same:

```

#include <iostream>
using namespace std;
template <class T, class U>
class A {
    T x;
    U y;
public:
    A(T x , U y) { cout << "Constructor Called" << endl;
                  cout<< x <<" " << y << endl;
    }
};

int main() {
    A<char, char> a('@', '$');
    A<int, double> b(1, 3.6);
    return 0;
}

```

**Output:**

```

Constructor Called
@ $
Constructor Called
1 3.6

```

## C++ friend Function and friend Classes

Data hiding is a fundamental concept of OOP. It restricts the access of private members from outside of the class.

Similarly, protected members can only be accessed by derived classes and are inaccessible from outside. For example,

```
class MyClass {
    private:
        int member1;
}
int main() {
    MyClass obj;

    // Error! Cannot access private members from here.
    obj.member1 = 5;
}
```

However, there is a feature in C++ called **friend functions** that break this rule and allow us to access member functions from outside the class. Similarly, there is a **friend class** as well.

### friend Function in C++

A friend function can access the private and protected data of a class. We declare a friend function using the **friend** keyword inside the body of the class.

```
// C++ program to demonstrate the working of friend function
#include <iostream>
using namespace std;
class Distance {
    private:
        int meter;
        friend int addFive(Distance); // friend function
    public:
        Distance() : meter(0) {}
};
```

```
// friend function definition
int addFive(Distance d) {
    //accessing private members from the friend function
    d.meter += 5;
    return d.meter;
}

int main() {
    Distance D;
    cout << "Distance: " << addFive(D);
    return 0;
}
```

### **Output**

Distance: 5

Here, `addFive()` is a friend function that can access both private and public data members. Though this example gives us an idea about the concept of a friend function, it doesn't show any meaningful use.

A more meaningful use would be operating on objects of two different classes. That's when the friend function can be very helpful. See the following Example:

```
// Add members of two different classes using friend functions
#include <iostream>
using namespace std;
// forward declaration
class ClassB;
class ClassA {
public:
    // constructor to initialize numA to 12
    ClassA() { numA =12;}
private:
    int numA;
    // friend function declaration
    friend int add(ClassA, ClassB);
};
```



```

class ClassB {
    public:
        // constructor to initialize numB to 1
        ClassB() { numB = 1;}
    private:
        int numB;
        // friend function declaration
        friend int add(ClassA, ClassB);
};

// access members of both classes
int add(ClassA objectA, ClassB objectB) {
    return (objectA.numA + objectB.numB);
}

int main() {
    ClassA objectA;
    ClassB objectB;
    cout << "Sum: " << add(objectA, objectB);
    return 0;
}

```

### **Output:**

Sum: 13

In this program, **ClassA** and **ClassB** have declared **add()** as a friend function. Thus, this function can access private data of both classes.

One thing to notice here is the friend function inside **ClassA** is using the **ClassB**. However, we haven't defined **ClassB** at this point.

```

// inside classA
friend int add(ClassA, ClassB);

```

For this to work, we need a forward declaration of ClassB in our program.

```

// forward declaration
class ClassB;

```

## **friend Class in C++**

We can also use a friend Class in C++ using the **friend** keyword.

When a class is declared a friend class, all the member functions of the friend class become friend functions.

*// C++ program to demonstrate the working of friend class*

```
#include <iostream>
```

```
using namespace std;
```

*// forward declaration*

```
class ClassB;
```

```
class ClassA {
```

```
    private:
```

```
        int numA;
```

*// friend class declaration*

```
        friend class ClassB;
```

```
    public:
```

*// constructor to initialize numA to 12*

```
        ClassA() {numA = 12;}
```

```
};
```

```
class ClassB {
```

```
    private:
```

```
        int numB;
```

```
    public:
```

*// constructor to initialize numB to 1*

```
        ClassB() {numB = 1;}
```

*// member function to add numA*

*// from ClassA and numB from ClassB*

```
    int add() {
```

```
        ClassA objectA;
```

```
        return objectA.numA + numB;
```

```
    }
```

```
};
```

```
int main() {
    ClassB objectB;
    cout << "Sum: " << objectB.add();
    return 0;
}
```

### **Output:**

Sum: 13

Since **ClassB** is a friend class, we can access all members of **ClassA** from inside **ClassB**. However, we cannot access members of **ClassB** from inside **ClassA**. It is because **friend relation in C++** is only granted, not taken.

So by using friend class, classes that are '**friends**' can access not just the public members, but the private and protected members too of the other class.

### **What is the difference between Inheritance and Friendship in C++?**

In C++, **friendship** is not inherited. If a base class has a **friend function**, then the function doesn't become a friend of the derived class(es).

### **المحاضرة الثالثة عشر والرابعة عشر العملية**

القيام بتطبيق أمثلة برمجية عن الـ **friend class** والـ **friend function**